

useVocalJitter.ts

frontend/hooks | TypeScript

```
/**  
 * useVocalJitter.ts  
 */  
  
 * Extracts vocal jitter – a real-time cortisol proxy – from the microphone stream.  
 *  
 * Jitter is the cycle-to-cycle variability in the fundamental frequency (F0) of  
 * the voice. Under psychological stress, cortisol tightens laryngeal muscles,  
 * introducing micro-instabilities in pitch that manifest as elevated jitter.  
 *  
 * Pipeline:  
 * Mic → Web Audio API → AnalyserNode → time-domain buffer →  
 * autocorrelation pitch detection → pitch period sequence → jitter → normalized score  
 *  
 * Returns a normalized jitter score (0-100) fed into the fusion model as the  
 * "audio" channel (weight: 0.5 – highest weight of the three channels).  
 */  
  
import { useEffect, useRef, useState, useCallback } from 'react';  
  
// ■■■■ Processing config ■■■■  
const SAMPLE_RATE = 44100;  
const BUFFER_SIZE = 2048; // ~46ms window for pitch detection  
const UPDATE_INTERVAL = 500; // Compute jitter every 500ms  
const MIN_FREQ = 80; // Hz – lowest detectable F0 (deep male voice)  
const MAX_FREQ = 300; // Hz – highest detectable F0 (high female voice)  
const JITTER_BASELINE = 0.8; // % relative jitter at rest (personal typical)  
const JITTER_MAX = 4.0; // % relative jitter at high stress (normalization ceiling)  
  
interface UseVocalJitterReturn {  
  jitterScore: number; // Normalized 0-100  
  isVoiceActive: boolean; // Whether speech is currently detected  
  jitterRaw: number; // Raw % relative jitter for display  
}  
  
export function useVocalJitter(): UseVocalJitterReturn {  
  const [jitterScore, setJitterScore] = useState(0);  
  const [isVoiceActive, setIsVoiceActive] = useState(false);  
  const [jitterRaw, setJitterRaw] = useState(0);  
  
  const audioCtxRef = useRef<AudioContext | null>(null);  
  const analyserRef = useRef<AnalyserNode | null>(null);  
  const pitchHistory = useRef<number[]>([]);  
  const intervalRef = useRef<ReturnType<typeof setInterval> | null>(null);  
  
  // ■■ Autocorrelation-based pitch detection ■■■■  
  // Returns the fundamental frequency in Hz, or 0 if no clear pitch found.  
  const detectPitch = useCallback((buffer: Float32Array): number => {  
    const minPeriod = Math.floor(SAMPLE_RATE / MAX_FREQ);  
    const maxPeriod = Math.floor(SAMPLE_RATE / MIN_FREQ);  
  
    let maxCorrelation = -Infinity;  
    let bestPeriod = 0;  
  
    for (let lag = minPeriod; lag <= maxPeriod; lag++) {  
      let correlation = 0;  
      for (let i = 0; i < buffer.length - lag; i++) {  
        correlation += buffer[i] * buffer[i + lag];  
      }  
  
      if (correlation > maxCorrelation) {  
        maxCorrelation = correlation;  
        bestPeriod = lag;  
      }  
    }  
  
    // Reject if correlation is too weak (unvoiced / silence)  
    const normalizedCorrelation = maxCorrelation / (buffer.length * 0.5);
```



```

}
}, UPDATE_INTERVAL);
} catch (err) {
console.error('[BioSync] Microphone access denied:', err);
}
};

init();

return () => {
if (intervalRef.current) clearInterval(intervalRef.current);
audioCtxRef.current?.close();
stream?.getTracks().forEach(t => t.stop());
};
}, [detectPitch, computeJitter, normalizeJitter]);
return { jitterScore, isVoiceActive, jitterRaw };
}

```